SENDBIRD AI AGENT · MESSENGER SDK

종료된 대화에서 새 대화 시작하기

종료된 대화의 **"대화 시작"** 버튼이
새 대화를 생성하도록 커스터마이징

대상 기능 · ConversationLayout.Footer 슬롯 교체
기준 버전 · @sendbird/ai-agent-messenger-react v1.45.0
문서 작성 · 2026-11-11

목차

1. 현상과 결론
2. SDK 기본 동작
   - 2.1 종료된 대화에 붙는 버튼
   - 2.2 왜 새 대화가 만들어지지 않는가
3. 해결 방법
   - 3.1 기본 버튼 끄기
   - 3.2 푸터 컴포넌트 구현
   - 3.3 메신저에 연결
4. 다중 대화 설정에 따른 분기
5. 동작 확인 결과
6. 구현 시 유의 사항

1 현상과 결론

> **현상** — 종료된 과거 대화에 들어가 하단의 **"대화 시작"** 을 누르면, 새 대화가 생성되지 않고 **이미 열려 있는 다른 대화로 이동**합니다.

결론: SDK 의 의도된 동작입니다. 버그가 아니며, 동작을 바꾸려면 **고객사에서 푸터를 커스터마이징**해야 합니다.

SDK 기본 버튼은 "새 대화를 만든다"가 아니라 **"지금 활성화된 대화로 데려간다"**는 의미로 구현되어 있습니다. 따라서 진행 중인 대화가 이미 있으면 그 대화로 이동하는 것이 정상입니다.

설정으로 가능한가	아니오 — 동작을 바꾸는 설정값은 없습니다.
필요한 작업	기본 버튼 비활성화(설정 1줄) + 푸터 컴포넌트 1개 추가
네이티브 수정	불필요 (Android / iOS 모두 변경 없음)
주의	**다중 대화 설정에 따라 호출할 함수가 다릅니다** (4장)

2 SDK 기본 동작

2.1 종료된 대화에 붙는 버튼

대화가 **종료 상태**가 되면, SDK 는 메시지 목록 끝에 "대화 시작" 버튼을 자동으로 붙입니다. 이 버튼의 노출 여부는 아래 설정으로 제어합니다.

```
config.conversation.list.isTalkToAgentViewEnabled   // 기본값 true
```

2.2 왜 새 대화가 만들어지지 않는가

버튼이 호출하는 함수가 `goToActiveConversation` 이기 때문입니다.

```
// SDK 내부 — 종료된 대화에 붙는 "대화 시작" 뷰
const TalkToAgentView = ({ isClosed }) => {
  const { stringSet } = useLocalizationContext();
  const { channelSource, goToActiveConversation } = useConversationContext();

  if (!channelSource.channel?.isAIAgent || !isClosed) return null;

  return (
    <Button {...conversation.startNewConversationButton({
      onActivate: goToActiveConversation    // ← 새 대화 생성이 아님
    }).props}>
      {stringSet.TALK_TO_AGENT}
```

```
    </Button>
  );
};
```

이름과 동작이 어긋나 보이는 이유

버튼 레이블은 "대화 시작"이지만, 내부 동작은 **"활성 대화로 이동"** 입니다. 열려 있는 대화가 없을 때는 결과적으로 새 대화가 열리므로 기대대로 보이지만, **이미 진행 중인 대화가 있으면** 그 대화로 이동합니다.

대화 목록 화면의 "대화 시작"과는 다릅니다

대화 목록 화면 하단의 "대화 시작"은 **다중 대화 설정을 보고 분기**하여 새 대화를 만들기도 합니다. 이번에 다루는 것은 **대화 화면 안(종료된 대화 하단)의 버튼**이며, 둘은 서로 다른 컴포넌트입니다.

3 해결 방법

기본 버튼을 끄고, 같은 자리에 **새 대화를 만드는 버튼**을 직접 그립니다. SDK 가 공개한 `ConversationLayout.Footer` 슬롯을 사용하므로 내부 구현에 의존하지 않습니다.

3.1 기본 버튼 끄기

```
<FixedMessenger
  config={{ conversation: { list: { isTalkToAgentViewEnabled: false } } }}
  ... />
```

3.2 푸터 컴포넌트 구현

`ClosedConversationFooter.tsx`

```
import { ConversationStatus } from '@sendbird/chat/aiAgent';
import {
  ConversationLayout,
  useConversationContext,
  useLocalizationContext,
  useMessengerContext,
  useMessengerSessionContext,
} from '@sendbird/ai-agent-messenger-react';
import { useState } from 'react';

const { Footer: DefaultFooter } = ConversationLayout.defaults.components;

export function ClosedConversationFooter() {
  const { channelSource, goToActiveConversation } = useConversationContext();
  const { aiAgentInfo, createConversation } = useMessengerSessionContext();
  const { language, countryCode, context } = useMessengerContext();
  const { stringSet } = useLocalizationContext();
  const [pending, setPending] = useState(false);
```

```
  const isClosed =
    channelSource.channel?.conversation?.status === ConversationStatus.CLOSED;

  const startNewConversation = async () => {
    setPending(true);
    try {
      if (aiAgentInfo.isMultipleActiveConversationsEnabled) {
        // 다중 대화 허용 → 새 대화를 생성합니다.
        await createConversation({
          aiAgentId: aiAgentInfo.userId,
          language,
          country: countryCode,
          context,
        });
      }
      // 단일 대화 설정이면 새로 만들지 않고 기존 활성 대화로 이동합니다.
      await goToActiveConversation();
    } finally {
      setPending(false);
    }
  };

  return (
    <>
      {isClosed && (
        <button type="button" disabled={pending} onClick={startNewConversation}>
          {stringSet.TALK_TO_AGENT}
        </button>
      )}
      {/* 입력창 등 기본 푸터는 그대로 유지합니다. */}
      <DefaultFooter />
    </>
  );
}
```

import 경로에 주의하세요

`ConversationStatus` 는 메신저 패키지가 아니라 **`@sendbird/chat/aiAgent`** 에서 가져와야 합니다. 메신저 패키지에서 import 하면 타입 검사는 통과하지만 **런타임에 "does not provide an export named 'ConversationStatus'" 오류**가 납니다. (문자열 `'closed'` 와 직접 비교해도 됩니다)

3.3 메신저에 연결

```
<FixedMessenger
  config={{ conversation: { list: { isTalkToAgentViewEnabled: false } } }}
  ...
>
  <FixedMessenger.ConversationChildren>
    <ConversationLayout.Footer component={ClosedConversationFooter} />
```

```
  </FixedMessenger.ConversationChildren>
</FixedMessenger>
```

> **슬롯 위치**
>
> 대화 화면 푸터이므로 **`FixedMessenger.ConversationChildren`** 안에 넣어야 합니다.
> `ConversationListChildren` (목록 화면)에 넣으면 적용되지 않습니다.

4 다중 대화 설정에 따른 분기

이 부분이 가장 중요합니다. 대시보드의 다중 대화 설정에 따라 호출해야 하는 함수가 달라집니다.

대시보드 설정	호출할 함수	결과
`Multiple active conversations` (기본값)	`createConversation()` → `goToActiveConversation()`	새 대화를 만들고 그 대화로 이동
`Single active conversation`	`goToActiveConversation()`	새로 만들지 않고 기존 활성 대화로 이동

단일 대화 설정에서는 **동시에 두 개의 활성 대화를 가질 수 없으므로** `createConversation()` 을 호출하면 안 됩니다.

설정값은 아래 값으로 읽을 수 있습니다. 3.2 의 코드는 이 값으로 분기하므로 **대시보드 설정을 바꿔도 코드를 수정할 필요가 없습니다.**

```
const { aiAgentInfo } = useMessengerSessionContext();
aiAgentInfo.isMultipleActiveConversationsEnabled   // boolean
```

5 동작 확인 결과

샘플 앱에 적용하고 실제로 확인한 결과입니다. (다중 대화 허용 설정 기준)

확인 항목	결과
진행 중인 대화	"대화 시작" 버튼 **미노출** (종료 상태에서만 표시)
대화 종료 후	"대화 시작" 버튼 표시, 입력창 비활성
"대화 시작" 클릭	**채널 URL 변경 = 새 대화 생성 확인**
클릭 후 상태	`closed` → `open`, 입력창 활성화
새 대화 내용	웰컴 메시지부터 새로 시작

기본 푸터(입력창)	변화 없음 — 그대로 유지

확인 완료

종료된 대화에서 "대화 시작"을 누르면 기존 대화로 이동하지 않고 **새 대화가 생성**되는 것을 채널 URL 변경으로 확인했습니다.

6 구현 시 유의 사항

중복 클릭 방지

`createConversation()` 은 네트워크 요청입니다. 응답 전에 다시 눌리면 **대화가 두 개 생성**될 수 있으므로, 위 코드처럼 `pending` 상태로 버튼을 비활성화해 주세요.

context 전달

새 대화는 **기존 대화의 context 를 물려받지 않습니다.** `createConversation()` 에 `context` 를 넘기지 않으면 사용자 식별 정보나 시간대 같은 값이 빠진 채 대화가 시작되므로, `useMessengerContext()` 에서 받은 값을 그대로 전달해 주세요.

버튼 레이블

`stringSet.TALK_TO_AGENT` 을 쓰면 SDK 의 다국어 설정을 그대로 따릅니다. 문구를 직접 지정하면 언어 전환 시 번역되지 않으니 주의하세요.

종료 상태 판별

`channelSource.channel?.conversation?.status` 로 확인합니다. **옵셔널 체이닝을 생략하지 마세요** — 대화가 아직 로드되지 않은 시점에는 `channel` 이 `undefined` 입니다.

본 문서는 @sendbird/ai-agent-messenger-react v1.45.0 기준으로 작성되었습니다. 2장의 SDK 기본 동작은 배포 번들 코드를 직접 확인한 내용이며, 5장의 확인 결과는 다중 대화 허용 설정에서 측정한 것입니다. 사용된 API 는 모두 SDK 가 공개적으로 export 하는 항목입니다. SDK 버전 업그레이드 시 동작을 재확인하시기 바랍니다.